Cursor Prompt — Rooftop Rain Pump Controller

Copy everything below the line into Cursor (Agent mode) in an empty repo. Work iteratively: let it scaffold, review, then say "continue".

Build a production-quality Python application called **pumpd** that automatically controls rooftop rain-drain pumps based on weather forecasts. It runs on an Ubuntu server via Docker Compose. The pumps are driven by Tuya (Smart Life) smart switches that also appear in SmartThings.

Tech stack (use exactly this)

- Python 3.12, FastAPI + Uvicorn, APScheduler, SQLAlchemy 2.x + SQLite, Pydantic v2 for config and models
- tinytuya for local LAN control of Tuya devices (primary control path)
- httpx for the SmartThings REST API (fallback control path) and weather APIs
- paho-mqtt for a rain-sensor input (Phase 2, build the interface now)
- pytest + pytest-asyncio; ruff + mypy; Dockerfile + docker-compose.yml
- Jinja2 server-rendered dashboard (no JS framework; htmx allowed for auto-refresh)

Repository layout

```
pumpd/
  app/
    main.py            # FastAPI app, lifespan starts scheduler
    config.py          # Pydantic Settings; loads config.yaml + .env for secrets
    models.py          # SQLAlchemy models
    scheduler.py       # APScheduler jobs
    weather/
      base.py          # WeatherProvider ABC -> HourlyForecast[]
      open_meteo.py    # primary, no API key
```

```
    nws.py            # secondary (US), no API key
  devices/
    base.py           # PumpDevice ABC: turn_on(), turn_off(), get_state()
    tuya_local.py     # tinytuya adapter (device_id, ip, local_key, version)
    smartthings.py    # cloud adapter (PAT token, device_id)
    composite.py      # tries tuya_local first, falls back to smartthings, verifies state after commands
  signals/
    base.py           # RainSignal ABC -> RainState(is_raining, rate_mm_h, confidence, source, ts)
    forecast_signal.py # derives RainState from stored forecast
    mqtt_signal.py    # Phase 2: subscribes to MQTT topic for tipping-bucket data
  engine.py           # rules engine (pure functions, fully unit-testable)
  safety.py           # max-runtime, cooldown, watchdog enforcement
  notify.py           # ntfy POST and/or SMTP
  web/                # routes + Jinja2 templates for dashboard
tests/
config.example.yaml
docker-compose.yml    # pumpd + mosquitto (mosquitto commented out until Phase 2)
Dockerfile
README.md             # setup incl. tinytuya wizard local-key extraction, SmartThings PAT creation
```

Configuration (config.yaml, validated by Pydantic)

```
location: { latitude: 0.0, longitude: 0.0 }

weather:

  poll_minutes: 30

  providers: [open_meteo, nws]

rules:

  evaluate_minutes: 10

  precip_probability_threshold: 70     # %

  precip_amount_threshold_mm: 2.0      # expected within lookahead

  lookahead_hours: 2

  post_rain_drain_minutes: 30

  duty_cycle: { enabled: false, on_minutes: 10, off_minutes: 20 }

safety:

  max_continuous_runtime_minutes: 60

  min_cooldown_minutes: 15

  watchdog_stale_forecast_hours: 3     # no fresh forecast -> pumps off + alert

pumps:

  - name: north_pump

    tuya: { device_id: "", ip: "192.168.1.50", local_key: "", version: 3.4 }

    smartthings: { device_id: "" }

notifications:
```

```
  ntfy: { enabled: true, url: "https://ntfy.sh", topic: "" }

mqtt:           # Phase 2

  enabled: false

  host: mosquitto

  topic: sensors/rain
```

Secrets (SmartThings PAT, Tuya local keys) may alternatively come from `.env`.

Behavior requirements

1. **Forecast job**: poll providers on schedule; store hourly precipitation probability + amount in SQLite; mark provider health.
2. **Rules evaluation** (every `evaluate_minutes`, pure function of inputs → decisions, so it's testable):
   - Start pumps when probability ≥ threshold AND amount ≥ threshold within lookahead.
   - Keep running (with optional duty cycle) while rain signal active.
   - After rain signal clears, run `post_rain_drain_minutes`, then stop.
   - If MQTT rain signal is enabled, it takes precedence over forecast for "is it raining now"; forecast still drives pre-emptive starts.
3. **Control path**: composite adapter tries Tuya local (retry ×3), falls back to SmartThings; after every command, read state back and verify; log + notify on mismatch or total failure.
4. **Modes per pump**: `auto` / `manual_on` / `manual_off`. Manual overrides automation and auto-reverts to auto after a configurable timeout (default 4 h).
5. **Safety** (hard invariants, enforced regardless of rules/manual): never exceed max continuous runtime; enforce cooldown; watchdog turns pumps off and alerts if forecasts are stale or the engine stops evaluating. All decisions and commands go to an `events` table.
6. **Dashboard** (`/`): pump cards (state, mode, runtime today, override buttons), current rain signal, next-12 h forecast bar, recent events. REST endpoints: `GET /api/status`, `POST /api/pumps/{name}/mode`, `GET /api/events`, `GET /health`.
7. **Startup/shutdown**: on startup, reconcile actual device states with DB; on clean shutdown, leave pumps in a safe state (off unless manual_on).

Quality bar

- Unit tests for the rules engine covering: pre-emptive start, duty cycle, post-rain drain, sensor-overrides-forecast, stale-forecast watchdog, max-runtime cutoff, manual override + revert. Mock all I/O.
- Type hints throughout; mypy and ruff clean.
- Structured logging (JSON option) with decision reasons ("started north_pump: 82% prob, 4.1mm expected in 2h").
- README with: docker compose quickstart, tinytuya wizard walkthrough to get local keys, SmartThings PAT setup, config reference, Phase 2 rain-sensor section (ESPHome tipping-bucket YAML example publishing to `sensors/rain`).

Start by scaffolding the repo, config models, and the rules engine with its tests. Then implement weather providers, device adapters, scheduler, safety, and finally the web dashboard.